

Embedded Software Coding Assignment

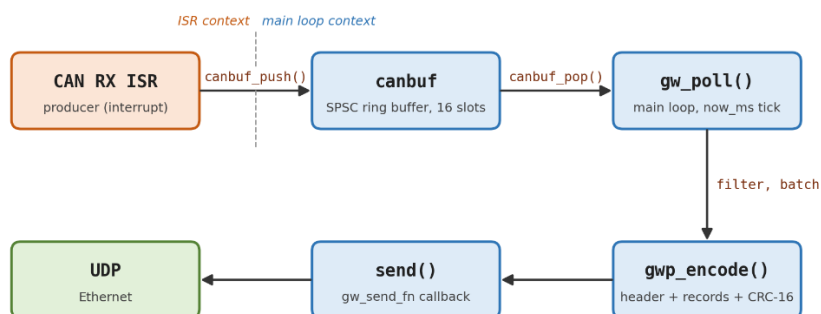
CAN-to-UDP Gateway Core · Candidate Task Sheet

Item	Details
Expected effort	approx. 90 minutes, please do not exceed 2 hours
Language / standard	C11, standard library only, host build (no hardware needed)
Submission	Link to a private Git repository (GitHub or GitLab) with collaborator access for steinke@attacan.com
Deadline	5 days from receiving the tasks
Contact for questions	Martin Steinke, steinke@attacan.com

1 Introduction

In this assignment you implement the hardware-independent core of a small gateway that receives CAN frames, filters them, bundles them into binary packets and hands the packets to a UDP transmit function. The code runs on your PC, but it is structured like microcontroller firmware: a lock-free buffer between interrupt and main loop, a byte-exact wire protocol with CRC, and a cooperative poll function driven by a millisecond tick.

The data flow is:



The API is fixed by three header files (Appendix A and the starter package). Your job is to implement them exactly according to this specification and to prove it with your own tests.

2 Ground rules

1. Create a Git repository and commit your work in meaningful steps. A single commit containing the finished solution is not what we are looking for.
2. Use C11 and the standard library only. No dynamic memory allocation (`malloc` etc.), no operating-system specific calls inside the modules.
3. Provide a `Makefile` or CMake project. `make test` (or the documented CMake equivalent) must build and run your tests. The modules must compile without warnings using `-std=c11 -Wall -Wextra -Werror` with GCC or Clang.
4. Do not change the API of the provided headers. The member layout of `canbuf_t` may be adapted (for example to use `_Atomic` types); everything else in the headers stays as provided.
5. Put the complete implementation into the three files `src/canbuf.c`, `src/gwproto.c` and `src/gateway.c`. Helper functions go into these files as `static` functions. These files must not contain `main()`.
6. Write your own unit tests in `test/`. No framework is required; plain `assert()` is fine.
7. Add a short `README.md`: how to build and test, your key design decisions, known limitations, and where you used AI tools (see section 3).

Required repository layout:

<code>include/</code>	<code>canbuf.h</code>	<code>gwproto.h</code>	<code>gateway.h</code>	(provided, API unchanged)
<code>src/</code>	<code>canbuf.c</code>	<code>gwproto.c</code>	<code>gateway.c</code>	(your implementation)
<code>test/</code>	your tests			
<code>Makefile</code>	(or <code>CMakeLists.txt</code>)			
<code>README.md</code>				

3 Use of AI tools and the follow-up interview

You may use AI assistants the same way you would use them at work: for individual functions, for looking things up, or for reviewing your code. Please do not generate the whole solution with an AI tool. State briefly in the README where you used one; this is not held against you.

After submission we invite you to a technical interview of about 60 minutes via video call. Please be prepared to:

- share your screen and build and run your submission from a fresh clone,
- walk us through the code and explain your design decisions,
- extend or modify the code live while explaining what you are doing and why. Looking things up is fine; please do it visibly on the shared screen.

Please have a working development environment (compiler, editor, Git) ready for the interview. The modification task builds directly on your own code.

4 Module 1: Ring buffer (canbuf)

The ring buffer decouples the CAN receive interrupt (producer) from the main loop (consumer). There is exactly one producer and exactly one consumer. The implementation must be safe for this usage **without disabling interrupts**.

Requirements

- `CANBUF_SIZE` is 16 and a power of two. **All 16 slots must be usable**: after 16 successful pushes without a pop the buffer is full.

- `canbuf_init()` resets the buffer to empty and the drop counter to 0.
- `canbuf_push()` copies the frame into the buffer and returns `true`. If the buffer is full, the **new frame is discarded**, the drop counter is incremented by one and the function returns `false` (policy "drop newest"). Already stored frames are never overwritten.
- `canbuf_pop()` copies the oldest frame to `*out` and returns `true`, or returns `false` if the buffer is empty.
- `canbuf_count()` returns the number of stored frames, `canbuf_drops()` the total number of discarded frames since init.
- The buffer must work correctly for an unlimited number of push/pop operations (index wrap-around).
- NULL pointer arguments: `push/pop` return `false` without counting a drop; `count/drops` return 0; `init` does nothing.

5 Module 2: Packet protocol (gwproto)

5.1 Packet layout

All multi-byte fields are transmitted in **big-endian** byte order. A packet contains 1 to 8 frame records.

Offset	Size	Field	Value / meaning
0	2	Magic	0x47 0x57 (ASCII "GW")
2	1	Version	0x01
3	1	Frame count N	1 ... 8
4	2	Sequence number	uint16, big-endian
6	13 · N	Frame records	see 5.2
6 + 13 · N	2	CRC	CRC-16 over all preceding bytes (offset 0 ... 5 + 13 · N), big-endian

Total packet length: $8 + 13 \cdot N$ bytes (21 bytes for $N = 1$, 112 bytes for $N = 8$). The macro `GWP_PKT_LEN(n)` computes this value.

5.2 Frame record (13 bytes)

Record offset	Size	Field	Meaning
0	4	CAN ID	Value of <code>can_frame_t.id</code> incl. IDE flag in bit 31, big-endian
4	1	DLC	0 ... 8
5	8	Data	Always 8 bytes. Bytes at index $\geq$ DLC are always 0x00 in the packet, regardless of the contents of <code>can_frame_t.data</code> .

5.3 Frame validity

`gwp_frame_valid()` returns `true` if and only if all of the following hold:

- `DLC` ≤ 8 ,
- bits 30 and 29 of the ID are 0 (`CAN_ID_RESERVED`),
- standard frame (bit 31 = 0): ID $\leq 0x7FF$; extended frame (bit 31 = 1): 29-bit ID $\leq 0x1FFFFFFF$ (implied by the reserved-bit rule).

5.4 CRC

CRC-16/CCITT-FALSE: polynomial 0x1021, initial value 0xFFFF, input and output not reflected, final XOR 0x0000. The check value for the ASCII string "123456789" is **0x29B1**. The CRC of zero bytes is 0xFFFF. A table-driven or bitwise implementation are both acceptable.

5.5 Encoding: `gwp_encode()`

On success the function writes the complete packet to `out` and returns its length. Otherwise it returns the error code of the **first** matching condition below, and the contents of `out` are unspecified.

Check order	Condition	Return value
1	<code>frames</code> or <code>out</code> is NULL, <code>n</code> = 0 or <code>n</code> > 8	<code>GWP_ERR_ARG</code>
2	<code>out_size</code> < <code>GWP_PKT_LEN(n)</code>	<code>GWP_ERR_LEN</code>
3	at least one frame is not valid (5.3)	<code>GWP_ERR_FRAME</code>

5.6 Decoding: `gwp_decode()`

On success the function stores the sequence number in `*seq`, the frames in `frames[0 ... N-1]` and returns `N`. Data bytes at index $\geq$ `DLC` are set to 0 in the output frames. Otherwise it returns the error code of the **first** matching condition below; in the error case the output parameters are unspecified.

Check order	Condition	Return value
1	<code>in</code> , <code>seq</code> or <code>frames</code> is NULL	<code>GWP_ERR_ARG</code>
2	<code>len</code> < 6	<code>GWP_ERR_LEN</code>
3	magic bytes are not 0x47 0x57	<code>GWP_ERR_MAGIC</code>
4	version is not 0x01	<code>GWP_ERR_VERSION</code>
5	<code>N</code> = 0 or <code>N</code> > 8	<code>GWP_ERR_COUNT</code>
6	<code>N</code> > <code>max_frames</code>	<code>GWP_ERR_ARG</code>
7	<code>len</code> $\neq$ <code>GWP_PKT_LEN(N)</code>	<code>GWP_ERR_LEN</code>
8	received CRC $\neq$ calculated CRC	<code>GWP_ERR_CRC</code>
9	at least one decoded frame is not valid (5.3)	<code>GWP_ERR_FRAME</code>

5.7 Test vectors

Vector 1: sequence number 0x0102, one standard frame with ID 0x123, DLC 3, data DE AD BE.

Seite 4/10

47 57 01 01 01 02	header
00 00 01 23 03 DE AD BE 00 00 00 00 00	record 0
D0 76	CRC (21 bytes total)

Vector 2: sequence number 0xFFFF, two frames: extended ID 0x18FF50E5 (id field 0x98FF50E5 incl. IDE flag), DLC 8, data 01 ... 08; standard ID 0x7FF, DLC 0.

47 57 01 02 FF FF	header
98 FF 50 E5 08 01 02 03 04 05 06 07 08	record 0
00 00 07 FF 00 00 00 00 00 00 00 00	record 1
DD AA	CRC (34 bytes total)

6 Module 3: Gateway

6.1 Initialisation and filters

- `gw_init()` stores the buffer pointer and the send callback and resets all other state: no filters, empty batch, sequence number 0.
- `gw_add_filter()` appends a filter and returns `true`, or returns `false` if `GW_MAX_FILTERS` (4) filters are already configured (or `gw` is NULL).
- A frame is **accepted** if for at least one configured filter `(id & mask) == (match & mask)` holds. The complete 32-bit `id` field including the IDE flag in bit 31 is used. If no filter is configured, every frame is accepted.

6.2 `gw_poll(gw, now_ms)`

`gw_poll()` is called periodically from the main loop. `now_ms` is a free-running millisecond counter that **wraps around** from 0xFFFFFFFF to 0. Each call performs these steps:

1. Pop frames from the buffer until it is empty. Discard every frame that is not valid (5.3) or not accepted by the filters. Append all other frames to the batch.
2. When a frame is added to an **empty** batch, record `now_ms` as the batch start time.
3. As soon as the batch contains 8 frames, send it immediately (one call may therefore produce several packets).
4. After the buffer is empty: if the batch is not empty and at least `GW_BATCH_TIMEOUT_MS` (10 ms) have elapsed since the batch start time, send it.

Sending means: encode the batch with the current sequence number, call `send(send_ctx, packet, length)`, increment the sequence number (wrapping from 0xFFFF to 0; the return value of `send` is ignored) and empty the batch.

Example: frames arrive at `t = 100 ms` and `t = 105 ms`. Calls at `t = 105` and `t = 109` send nothing; the call at `t = 110` sends one packet with both frames.

7 How your submission is assessed

- Correctness against this specification, including all edge cases and error codes.
- Code quality: readability, suitability for a microcontroller (memory, determinism, portability, concurrency), no undefined behaviour.
- Quality and meaningfulness of your own tests.
- Git history and README.
- In the interview: your ability to explain and extend your own code.

If you run out of time, submit what you have and describe open points in the README. A clean, well-tested partial solution is better than a complete but unverified one.

Appendix A Provided header files

The same files are included in the starter package. Copy them unchanged into `include/`.

`include/canbuf.h`

```
/* canbuf.h - Single-producer / single-consumer ring buffer for CAN frames.

 * Provided header: the API must not be changed. The member layout of
 * canbuf_t may be adapted (e.g. to use _Atomic types). */

#ifndef CANBUF_H
#define CANBUF_H

#include <stdint.h>
#include <stdbool.h>

#define CAN_ID_IDE      0x80000000u /* bit 31: extended (29-bit) identifier */
#define CAN_ID_RESERVED 0x60000000u /* bits 30..29: must be zero */
#define CAN_ID_EXT_MASK 0x1FFFFFFFu
#define CAN_ID_STD_MASK 0x000007FFu

typedef struct {
    uint32_t id;      /* identifier incl. IDE flag in bit 31 */
    uint8_t  dlc;      /* 0..8 */
    uint8_t  data[8];
} can_frame_t;

#define CANBUF_SIZE 16u /* power of two, all slots usable */

typedef struct {
    can_frame_t    slots[CANBUF_SIZE];
    volatile uint32_t head; /* written by producer only */
    volatile uint32_t tail; /* written by consumer only */
    volatile uint32_t drops; /* written by producer only */
} canbuf_t;

void    canbuf_init(canbuf_t *b);
bool    canbuf_push(canbuf_t *b, const can_frame_t *f); /* ISR context */
bool    canbuf_pop(canbuf_t *b, can_frame_t *out);      /* main context */
uint32_t canbuf_count(const canbuf_t *b);
uint32_t canbuf_drops(const canbuf_t *b);
```

```
#endif /* CANBUF_H */
```

include/gwproto.h

```
/* gwproto.h - Binary gateway protocol (CAN frames over UDP).

 * Provided header: must not be changed. */

#ifndef GWPROTO_H
#define GWPROTO_H

#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include "canbuf.h"

#define GWP_MAGIC0      0x47u
#define GWP_MAGIC1      0x57u
#define GWP_VERSION     0x01u
#define GWP_MAX_FRAMES  8u
#define GWP_HDR_LEN     6u
#define GWP_REC_LEN     13u
#define GWP_CRC_LEN     2u
#define GWP_PKT_LEN(n)  (GWP_HDR_LEN + GWP_REC_LEN * (n) + GWP_CRC_LEN)
#define GWP_MAX_LEN     GWP_PKT_LEN(GWP_MAX_FRAMES)

typedef enum {
    GWP_OK          = 0,
    GWP_ERR_ARG     = -1,
    GWP_ERR_LEN     = -2,
    GWP_ERR_MAGIC   = -3,
    GWP_ERR_VERSION = -4,
    GWP_ERR_COUNT   = -5,
    GWP_ERR_FRAME   = -6,
    GWP_ERR_CRC     = -7
} gwp_err_t;

/* CRC-16/CCITT-FALSE: poly 0x1021, init 0xFFFF, no reflection, no xorout */
uint16_t gwp_crc16(const uint8_t *data, size_t len);

/* true if the frame is valid according to the protocol rules */
bool gwp_frame_valid(const can_frame_t *f);
```

```

/* Returns number of bytes written (> 0) or a negative gwp_err_t. */
int gwp_encode(uint16_t seq, const can_frame_t *frames, size_t n,
               uint8_t *out, size_t out_size);

/* Returns number of decoded frames (> 0) or a negative gwp_err_t. */
int gwp_decode(const uint8_t *in, size_t len, uint16_t *seq,
               can_frame_t *frames, size_t max_frames);

#endif /* GWPROTO_H */

```

include/gateway.h

```

/* gateway.h - CAN to UDP gateway core.

 * Provided header: the API must not be changed. */

#ifndef GATEWAY_H
#define GATEWAY_H

#include <stddef.h>
#include <stdint.h>
#include <stdbool.h>
#include "canbuf.h"
#include "gwproto.h"

#define GW_MAX_FILTERS      4u
#define GW_BATCH_TIMEOUT_MS 10u

/* Transmit hook (e.g. mapped to a UDP send). Return value is informational. */
typedef int (*gw_send_fn)(void *ctx, const uint8_t *pkt, size_t len);

typedef struct {
    uint32_t match;
    uint32_t mask;
} gw_filter_t;

typedef struct {
    canbuf_t    *rx;
    gw_send_fn   send;
    void         *send_ctx;
    gw_filter_t  filters[GW_MAX_FILTERS];
    size_t       n_filters;
    can_frame_t  batch[GWP_MAX_FRAMES];

```

Seite 9/10

```
    size_t      batch_n;

    uint32_t    batch_start_ms;

    uint16_t    seq;
} gw_t;

void gw_init(gw_t *gw, canbuf_t *rx, gw_send_fn send, void *send_ctx);

bool gw_add_filter(gw_t *gw, uint32_t match, uint32_t mask); /* false if full */

void gw_poll(gw_t *gw, uint32_t now_ms);

#endif /* GATEWAY_H */
```